

Unit-3

❖ Understanding Controls

- In **C# GUI programming**, a **control** is any UI element (object/component) that allows interaction between the user and the application. Controls are part of the **.NET Framework Class Library** under the **System.Windows.Forms** (for WinForms) or **System.Web.UI.WebControls** (for ASP.NET) or **System.Windows.Controls** (for WPF).
- C#, "controls" refer to reusable components that provide specific user interface (UI) or functionality elements within a web page. These controls abstract away the complexities of HTML, JavaScript, and server-side processing, allowing developers to build web pages more efficiently.
- **HTML Server Controls:** These are standard HTML elements (e.g., <div>, <input type="text">, <button>) that are marked with runat="server" and an id attribute, enabling them to be accessed and manipulated programmatically on the server-side using C#. `<input type="text" id="txtName" runat="server" />`
- **Web Server Controls:** These are richer, more powerful controls provided by ASP.NET, specifically designed for web development. They offer a wide range of functionalities and often encapsulate complex UI elements and behaviors. Examples include TextBox, Button, Label, GridView, Calendar, and ValidationSummary. `<asp:Button ID="btnSubmit" runat="server" Text="Submit" OnClick="btnSubmit_Click" />`
- **What is Control?**
- A **control** is a graphical object placed on a form (or window/web page) that enables users to input, display, or interact with data. For Example: Buttons, Labels, TextBoxes, CheckBoxes, ComboBoxes, DataGridView, etc.

❖ Net common controls: -

- **.NET common controls in C#** (mainly used in **Windows Forms applications**).

Control	Description	Common Properties	Important Events	Example Use
Label	Displays read-only text.	Text, ForeColor, BackColor, Font	—	Show static text like "Username".
TextBox	Accepts single/multiline text input.	Text, Multiline, MaxLength, PasswordChar	TextChanged	Enter user input (e.g., name, email).
Button	Triggers an action when clicked.	Text, Enabled, BackColor	Click	Submit form, perform an action.
CheckBox	Lets user select multiple choices.	Checked, Text	CheckedChanged	Accept terms, choose multiple items.
RadioButton	Allows one selection in a group.	Checked, Text	CheckedChanged	Select gender (Male/Female).

Control	Description	Common Properties	Important Events	Example Use
ComboBox	Dropdown list for selection.	Items, SelectedIndex, Text	SelectedIndexChanged	Choose country from a list.
ListBox	Displays list of items.	Items, SelectionMode	SelectedIndexChanged	Select multiple hobbies.
PictureBox	Displays an image.	Image, SizeMode	—	Show logo, photo.
ProgressBar	Shows progress of a task.	Minimum, Maximum, Value	—	Show file download progress.
DateTimePicker	Allows date/time selection.	Value, Format, MinDate, MaxDate	ValueChanged	Pick DOB or appointment date.
DataGridView	Displays tabular data.	DataSource, Rows, Columns	CellClick, CellValueChanged	Show database records.
GroupBox	Groups related controls.	Text, Controls	—	Group gender RadioButtons.
Panel	Container for other controls.	Controls, BackColor	—	Place controls in a scrollable area.
TabControl	Provides tabbed pages.	TabPage, SelectedIndex	SelectedIndexChanged	Settings screen with tabs.
RichTextBox	Advanced TextBox with formatting.	Text, SelectionFont, SelectionColor	TextChanged	Write formatted notes.
TrackBar	Slider for numeric values.	Minimum, Maximum, Value	Scroll	Adjust volume or brightness.
ToolTip	Provides hints when hovering.	ToolTipTitle	—	Show info when hovering over a button.
MenuStrip	Creates menus at top of form.	Items	Click	File → Open, Save, Exit.
StatusStrip	Shows status messages at bottom.	Items, Text	—	Show app status or progress.
TreeView	Displays hierarchical data.	Nodes	AfterSelect	File explorer structure.
ListView	Displays list with icons/details.	View, Items, Columns	SelectedIndexChanged	File explorer (Details, Tiles).

-

❖ Control Hierarchy

- Web Forms, controls are organized in a hierarchical structure, forming a control tree. This hierarchy is fundamental to how pages are processed and how controls interact.
- The hierarchy establishes a clear parent-child relationship. A control's Parent property points to its immediate container control, while its child controls are accessible through its Controls collection. This allows for programmatic

traversal of the control tree.

- This hierarchical structure provides a structured and organized way to manage and interact with the various elements that constitute an ASP.NET Web Form.

❖ **Label And Link Label**

- In VB.NET Windows Forms applications, both Label and LinkLabel are controls used to display text, but they serve different purposes.
- A **Label** control is a static display element used to present non-interactive text to the user. Its primary function is to provide descriptive information or instructions on a form.
- **Static Text:** It displays plain text that does not respond to user interaction in a meaningful way beyond basic display.
- **Properties:** Common properties include Text, Font, ForeColor, BackColor, TextAlign, etc., for controlling its appearance and content.
- A **LinkLabel** control is a specialized version of the Label control that enables the creation of clickable hyperlinks within a Windows Forms application. It combines the text display capabilities of a Label with the interactive functionality of a hyperlink.
- It allows you to define specific parts of its text as clickable links.
- This **property** is used to define the starting position and length of the text within the LinkLabel that should function as a link.
- This event is triggered when a user clicks on the designated link area. You can write code within this event handler to perform actions like opening a web page, launching an application, or navigating to another form.
- It includes **properties** like LinkColor, VisitedLinkColor, and ActiveLinkColor to customize the appearance of the link in different states.
- Often used within the LinkClicked event to open URLs or files, typically using System.Diagnostics.Process.Start("URL_or_File_Path").
-

❖ **Text box and Rich Text box**

- Its primary function is to enable users to enter single or multiple lines of text, and it can also be used to display static or dynamic text within an application.
- **Properties:** Text box controls have various properties that can be set at design-time (in Visual Studio's Properties window) or programmatically at runtime.
- **Text:** Gets or sets the actual text content of the text box. **Multiline:** - Determines if the text box can display multiple lines of text. **Scrollbars:** controls the appearance of scrollbars when Multiline is true and text exceeds the visible area. **Password char:** Specifies a character to mask input for password fields. **Read only:** the text box display-only, preventing user input. **Max Length:** Limits the number of characters a user can type.
- **Rich text box:** The RichTextBox control is a powerful tool for displaying, entering, and manipulating text with various formatting options. It extends the functionality of a standard TextBox control by allowing features typically found in word processing applications. **Properties** of the RichTextBox in the Properties window, such as Text, Font, BackColor, ForeColor, WordWrap, ScrollBars, etc.

❖ **Picture Box**

- The PictureBox control is used for displaying images on the form. The Image property of the control allows you to set an image both at design time or at run time.
- **Properties:** **Allow Drop:** Specifies whether the picture box accepts data that a user drags on it. **Error Image:** Gets or specifies an image to be displayed when an error occurs during the image-loading process or if the image load is cancelled. **Image:** Gets or sets the image that is displayed in the control. **Image Location:** Gets or sets the path or the URL for the image displayed in the control. **Text:** Gets or sets the text for the picture box.

❖ Button

- Buttons can be added to a form or container control (like a Panel or GroupBox) either by dragging and dropping from the Toolbox in the Visual Studio designer or programmatically through code.
- **Properties:** Buttons have various properties that can be set to customize their appearance and behavior. **Text:** The text displayed on the button. **Name:** The unique identifier for the button, used to reference it in code (e.g., btnSubmit). **Enabled:** Determines if the button can be interacted with by the user (True/False). **Visible:** Determines if the button is displayed on the form (True/False). **Flat style:** Controls the visual style of the button (e.g., Standard, Flat, PopUp, System). **Image:** Allows adding an image to the button.

❖ Group Box

- It creates a framed area with an optional caption (set by the Text property) around a collection of related controls, enhancing the user interface's clarity and organization.
- It enables programmatic grouping, particularly useful for controls like RadioButtons, where selecting one option within a GroupBox automatically deselects others within the same GroupBox, allowing for independent selections across multiple GroupBoxes.
- How to use a group box: **Add to form** Drag and drop a GroupBox control from the Toolbox onto your Windows Form. **Add Controls:** Place other controls (e.g., RadioButtons, TextBoxes, CheckBoxes) inside the GroupBox by dragging them directly into its boundaries. **Set caption:** Modify the Text property of the GroupBox in the Properties window to provide a descriptive caption for the grouped controls.

❖ Panel

- The Panel control works as a container for other controls on the page. It controls the appearance and visibility of the controls it contains. It also allows generating controls programmatically.
- **Properties:** BackImageUrl -URL of the background image of the panel. **DefaultButton:** Gets or sets the identifier for the default button that is contained in the Panel control. **Direction:** Text direction in the panel. **Horizontal Align:** Horizontal alignment of the content in the panel. **ScrollBars:** Specifies visibility and location of scrollbars within the panel.

❖ Check Box and Radio button

- CheckBox control is a web server control used to allow users to select one or more options from a list of choices. It visually appears as a small square that can be checked (ticked) or unchecked.
- **Properties:** Checked: A boolean property that indicates whether the checkbox is currently selected (true) or unselected (false). This is the primary property for determining user selection. **AutoPostBack:** When set to true, the page automatically posts back to the server when the checkbox's Checked state changes. This enables immediate server-side processing of the selection. **Enabled:** Determines whether the checkbox is interactive (true) or disabled (false). **Visible:** Controls the visibility of the checkbox on the web page.
- **Radio button:** Web Forms using C#, the RadioButton control allows users to select a single option from a group of mutually exclusive choices. **Properties:** **Id, Text, Checked:** A boolean property indicating whether the radio button is selected (true) or not (false). **Group name:** This is crucial for creating mutually exclusive selections. All RadioButton controls that should behave as a single group (where only one can be selected at a time) must have the same Group

Name value. **Autopostback**: If set to true, the page will post back to the server immediately when the Checked state of the radio button changes. **CheckedChange**: An event that fires when the Checked state of the radio button changes. You can attach an event handler to this to perform actions when a selection is made.

❖ **List Box**

- The **ListBox** control is used to show multiple elements in a list, from which a user can select one or more elements, and the elements are generally displayed in multiple columns. The **ListBox class in C#** is used to represent the Windows list box and also provides different types of properties, methods, and events.
- **SelectionMode: Single (Default)**: Only one item can be selected at a time. Multiple: Allows users to select multiple items by holding down the Ctrl or Shift key while clicking. For SelectionMode="Single", SelectedItem and SelectedIndex properties can be used to get the selected item's text, value, and index. For SelectionMode="Multiple", you need to iterate through the Items collection and check the Selected property of each ListItem to identify selected items.

❖ **Checked List Box and Combo Box**

- Web Forms provides a way to display a group of checkboxes that can be dynamically generated and allows for multiple selections.
- **RenderingCheckboxlist**:
The Check Box List control renders a list of individual checkboxes within a single control. **RepeatDirection**: Determines whether the items are displayed Vertical (default) or Horizontal. **RepeatLayout**: Controls how the list is rendered (e.g., Table, Flow, UnorderedList, OrderedList). **RepeatColumns**: Specifies the number of columns to display the items in.
- **Combo Box**: In Windows Forms, the **ComboBox control** combines the features of a TextBox and a ListBox. It displays one item at a time, with additional items accessible through a drop-down menu.
- **DropDownHeight**: This property is used to set the height in pixels of the drop-down portion of the ComboBox control. **DropDownStyle**: This property is used to set a value specifying the style of the ComboBox control. **SelectedItem**: This property is used to set the currently selected item in the ComboBox.

❖ **Month Calendar and date time picker**

- Windows Forms applications, the MonthCalendar control provides a graphical interface for users to view and select date information.
- Drag and drop the MonthCalendar control from the Toolbox onto your Windows Form. Users can select a single date by clicking on it.
- They can also select a range of dates by clicking and dragging, or by holding down Shift and clicking on the start and end dates.
- A **GUI control** in Windows Forms that displays a calendar. Let's users **select dates, ranges of dates, or multiple dates**. Commonly used in applications for scheduling, booking, or selecting a date.
- Properties of MonthCalendar:- **MaxSelectionCount** → Maximum number of days the user can select. **SelectionStart** → First date in the selection. **SelectionEnd** → Last date in the selection. **TodayDate** → The date marked as "Today". **ShowToday** → Shows/hides today's date at the bottom. **ShowWeekNumbers** → Displays week numbers.

- **DatetimePicker:** A Windows Forms control used to **select a date and/or time**. Displays a **drop-down calendar** for date selection or a **time format** for time selection. Very useful in forms for choosing **DOB, booking dates, appointment times**, etc.
- The DateTimePicker control allows selecting a date and time by editing the displayed values in the control.
- If you click the arrow in the DateTimePicker control, it displays a month calendar, like a combo box control.
- The user can make selection by clicking the required date. The new selected value appears in the text box part of the control.
- **Properties of DateTimePicker:** **Value** → Gets or sets the selected date/time. **Format** → Sets how the date/time is displayed (*Short, Long, Time, or Custom*). **CustomFormat** → Allows you to define a custom display format (e.g., "dd/MM/yyyy hh:mm tt"). **MinDate / MaxDate** → Restricts the selectable date range. **ShowUpDown** → Displays up-down buttons instead of a drop-down calendar (useful for time).

❖ **Tree view and List view**

- A **hierarchical control** that displays data in a **tree-like structure**. Each item is called a **Node**. Nodes can contain **child nodes** (like folders and subfolders in Windows Explorer). Commonly used for **menus, file explorers, organization charts**, etc.
- Properties: **Nodes** → Collection of nodes in the TreeView. **SelectedNode** → Gets the currently selected node. **CheckBoxes** → Shows checkboxes next to nodes (if *True*). **ImageList** → Allows you to add icons to nodes.
- **List view:** The ListView control provides a way to display a collection of items in various visual formats. It is commonly used in Windows Forms applications to present data in a structured and user-friendly manner, similar to how files and folders are displayed in Windows Explorer.
- The ListView control is used to display a list of items. Along with the TreeView control, it allows you to create a Windows Explorer like interface.
- The Item property of the ListView control allows you to add and remove items from it. The **SelectedItem** property contains a collection of the selected items. The **Multiselect** property allows you to set select more than one item in the list view. **View** → Sets how items are displayed (*LargeIcon, SmallIcon, List, Tile, Details*). **Items** → Collection of items in the ListView. **Items** → Collection of items in the ListView. **FullRowSelect** → Highlights the entire row (useful in Details view). **GridLines** → Shows grid lines in Details view. **Columns** → Used when *View = Details* (to show columns).

❖ **Timer**

- the Timer control (specifically *System.Windows.Forms.Timer* for Windows Forms applications) is used to execute code at specified intervals.
- A **non-visual component** that executes code at **regular time intervals**.
- Properties: **Enabled** → Starts/stops the timer (*True* = running). **Interval** → Time (in milliseconds) between *Tick* events (Default = 100 ms).

❖ **Track bar and progress bar**

- The track bar control Windows Forms control used for navigating through a range of values or visually adjusting a numeric setting. It is often referred to as a "slider" control.
- The TrackBar control is useful when you need to allow users to select a numeric value within a specific range without requiring text input and validation.
- For Examples include adjusting volume levels, brightness settings, or controlling animation speeds. You can drag and drop the TrackBar control

onto a Windows Form in the Visual Studio designer and then configure its properties in the Properties window. You can also interact with it programmatically through its properties and events.

- Properties: **Minimum** → Lowest value (default = 0). **Maximum** → Highest value (default = 10). **Value** → Current position of the slider (default = 0). **TickFrequency** → Distance between ticks. **Orientation** → Horizontal (default) or Vertical. **SmallChange** / **LargeChange** → Amount the value changes with arrow keys/PageUp/PageDown.
- **Progress bar:** A progress bar control in VB.NET provides visual feedback to the user about the progress of a lengthy operation. It indicates that an application is actively performing a task and remains responsive.
- It represents a Windows progress bar control. It is used to provide visual feedback to your users about the status of some task. It shows a bar that fills in from left to right as the operation progresses.
- Properties: **Minimum, Maximum, Value, Step** → The amount increased when `PerformStep()` is called. **Style** → Determines how progress is displayed: Blocks, Continuous, Marquee.

❖ Image list control

- The **ImageList** is a special control in Windows Forms that stores a **collection of images**(icons,bitmaps,etc.).It is **not visible at runtime**; instead, it provides images to other controls such as ListView, TreeView, Toolbar / ToolStrip, Buttons / PictureBox.
- the ImageList component is a non-visual control used to store a collection of images, typically icons or small bitmaps, that can then be used by other controls on a Windows Forms application. It does not appear on the form at runtime.
- Properties: `ImageSize` → default is 16×16 (you can change to 32×32, etc.) `ColorDepth` → depth of image color.

❖ HScrollBar

- The **HScrollBar** (Horizontal ScrollBar) is used to select a value from a range by sliding a scroll box (thumb) horizontally.
- scrolling functionality can be implemented in various ways, primarily through the use of HScrollBar (Horizontal Scroll Bar) and VScrollBar (Vertical Scroll Bar) controls, or by leveraging the AutoScroll property of container controls like Panel or Form.
- Properties: **Minimum, Maximum, Value, LargeChange** → The amount increased/decreased when clicking the track area (default = 10). **SmallChange** → The amount increased/decreased when pressing arrow keys (default = 1).

❖ VScrollBar

- The **VScrollBar** control provides a vertical scroll bar that allows users to navigate through content that extends beyond the visible area of a control or container.
- The ScrollBar controls display vertical and horizontal scroll bars on the form. This is used for navigating through large amount of information. There are two types of scroll bar controls **HScrollBar** for horizontal scroll bars and **VScrollBar** for vertical scroll bars. These are used independently from each other.
- Properties: **Minimum, Maximum, Value, LargeChange** → The amount increased/decreased when clicking the track area (default = 10). **SmallChange** → The amount increased/decreased when pressing arrow keys (default = 1).

❖ **Tab Control**

- The **TabControl** is a container control used in Windows Forms applications to organize and display multiple sets of related information within a single form. It functions like the dividers in a notebook or labels in a filing cabinet, allowing users to switch between different "pages" or "tabs," each containing its own set of controls and content.
- **VB.NET Windows Forms**, the **TabControl** is a container control that allows you to create tabbed interfaces, similar to the tabs you see in browsers. Each **TabPage** inside the **TabControl** can contain its own set of controls (textboxes, buttons, labels, etc).
- **Add TabControl**: Open your Windows Forms Application in Visual Studio. From the **Toolbox**, drag and drop a **TabControl** onto your form. **Add TabPages**: Right-click the **TabControl** → **Add Tab** (you can add multiple tabs). Each **TabPage** acts like a separate panel where you can place controls. **Customize TabPages**: You can rename the tabs using the **Text** property in the Properties window.
- **Properties: TabPages** → Collection of tabs. **SelectedIndex** → Gets or sets the active tab. **SelectedTab** → Gets or sets the current **TabPage**. **Appearance** → Normal, FlatButtons, or Buttons style. **Alignment** → Position of tabs (Top, Bottom, Left, Right). **Multiline** → Allows multiple rows of tabs.

❖ **Common Dialog Controls:**

- There are many built-in dialog boxes to be used in Windows forms for various tasks like opening and saving files, printing a page, providing choices for colors, fonts, page setup, etc., to the user of an application. These built-in dialog boxes reduce the developer's time and workload.
- All of these dialog box control classes inherit from the Common Dialog box class and override the **RunDialog()** function of the base class to create the specific dialog box.
- The **ShowDialog** method is used to display all the dialog box controls at run-time. It returns a value of the type of **DialogResult**.
- **Abort** – returns **DialogResult.Abort** value, when user clicks an Abort button
- **Cancel** – returns **DialogResult.Cancel**, when user clicks a Cancel button.
- **Ignore** – returns **DialogResult.Ignore**, when user clicks an Ignore button.
- **No** – returns **DialogResult.No**, when user clicks a No button.
- **None** – returns nothing and the dialog box continues running.
- **OK** – returns **DialogResult.OK**, when user clicks an OK button.
- **Retry** – returns **DialogResult.Retry**, when user clicks an Retry button.
- **Yes** – returns **DialogResult.Yes**, when user clicks an Yes button.

❖ **Color Dialog control**

- The **ColorDialog** control is used to allow the user to pick a color from a standard color picker dialog box. Once the user selects a color, you can apply it to controls like a **Form**, **Label**, **TextBox**, etc.
- It represents a common dialog box that displays available colors along with controls that enable the user to define custom colors.

❖ **Folder Browser Dialog**

- The **FolderBrowserDialog** component in VB.NET and C# (Windows Forms) provides a modal dialog box for users to browse and select a folder.
- The **FolderBrowserDialog** lets the user pick a folder from their system in both **VB.NET** and **C#**.

❖ **Font Dialog**

- The **FontDialog** control is used to let the user pick a font (family, size, style, etc.) and then apply it to text in a control (like a `Label`, `TextBox`, or `RichTextBox`).
- It prompts the user to choose a font from among those installed on the local computer and lets the user select the font, font size, and color.

❖ **Open File Dialog**

- The **OpenFileDialog** control in Windows Forms is used when you want the user to browse and select a file from their computer. It supports filters (like `.txt`, `.jpg`, `.pdf`, etc.), multiselect, and returns the file path.
- It prompts the user to open a file and allows the user to select a file to open.

❖ **Save File Dialog**

- The **SaveFileDialog** control lets the user specify a location and filename to save a file. It works very much like `OpenFileDialog`, but for saving instead of opening.
- It prompts the user to select a location for saving a file and allows the user to specify the name of the file to save data.

❖ **MDI-Forms**

- MDI Stand for Multiple Document Interface.
- An **MDI application** allows you to have a parent form (container) that holds multiple child forms inside it (like how Microsoft Word or Excel used to work with multiple documents in one window).
- The foundation of a Multiple-Document Interface (MDI) application is the MDI parent form. This is the form that contains the MDI child windows, which are the sub-windows wherein the user interacts with the MDI application.
- Creating an MDI parent form is easy, both in the Windows Forms Designer and programmatically.
- In the **Properties** window, set the [IsMdiContainer](#) property to **true**.
-

❖ **Exploring Properties**

- An object is a type of user interface element you create on a Visual Basic form by using a toolbox control. In fact, in Visual Basic, the form itself is an object. Every Visual Basic control consists of three important elements.
- All the Visual Basic Objects can be moved, resized or customized by setting their properties. A property is a value or characteristic hold by a Visual Basic object, such as `Caption` or `ForeColor`.
- Properties can be set at design time by using the Properties window or at run time by using statements in the program code.

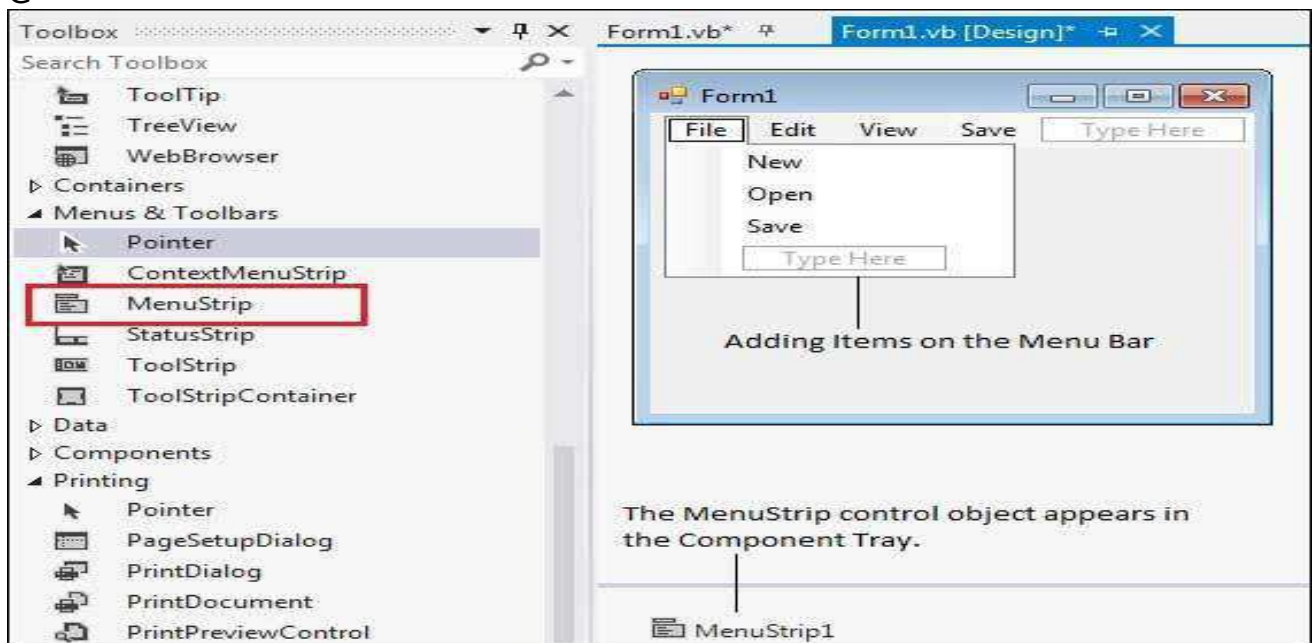
❖ **Methods and Events**

- **Events** are basically a user action like key press, clicks, mouse movements, etc., or some occurrence like system generated notifications. Applications need to respond to events when they occur.
- Clicking on a button, or entering some text in a text box, or clicking on a menu item, all are examples of events. An event is an action that calls a function or may cause another event. Event handlers are functions that tell how to respond to an event.

- A **method** is a block of code that performs a specific task. It is defined inside a class and can be **called (invoked)** when needed.
- A **method** is a block of code which only runs when it is called. You can pass data, known as parameters, into a method.
- Methods are used to perform certain actions, and they are also known as **functions**.
- A method is defined with the name of the method, followed by parentheses **()**. C# provides some pre-defined methods, which you already are familiar with, such as Main(), but you can also create your own methods to perform certain actions.

❖ Menubar

- The MenuStrip control represents the container for the menu structure.
- The MenuStrip control works as the top-level container for the menu structure. The ToolStripMenuItem class and the ToolStripDropDownMenu class provide the functionalities to create menu items, sub menus and drop-down menus.
- G



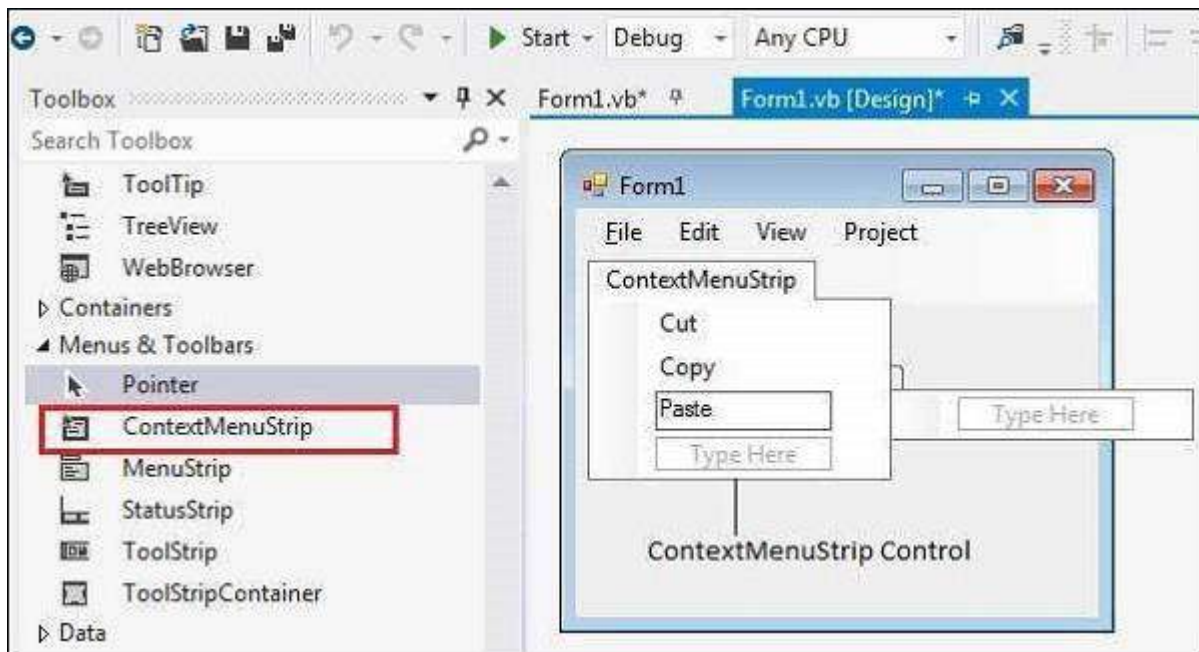
- **Properties:** -
- CanOverflow:- Gets or sets a value indicating whether the MenuStrip supports overflow functionality.
- GripStyle:- Gets or sets the visibility of the grip used to reposition the control.
- MDIWindowListItem: - Gets or sets the ToolStripMenuItem that is used to display a list of Multiple-document interface (MDI) child forms.
- ShowItemToolTips:- Gets or sets a value indicating whether ToolTips are shown for the MenuStrip.
- Stretch: - Gets or sets a value indicating whether the MenuStrip stretches from end to end in its container.
- **Method:** -
- MenuActivate: Occurs when the user accesses the menu with the keyboard or mouse.
- MenuDeactivate:- Occurs when the MenuStrip is deactivated.

❖ ContextMenuStrip

- The **ContextMenuStrip** control represents a shortcut menu that pops up over

controls, usually when you right click them. They appear in context of some specific controls, so are called context menus. For example, Cut, Copy or Paste options.

- This control associates the context menu with other menu items by setting that menu item's ContextMenuStrip property to the ContextMenuStrip control you designed.
- Context menu items can also be disabled, hidden or deleted. You can also show a context menu with the help of the Show method of the ContextMenuStrip control.
- **Properties:-**
- **SourceControl:-** Gets the last control that displayed the ContextMenuStrip control.



❖ Message Box

- The Message Box displays a message box and waits for the user to click a button and then an action is performed based on the button clicked by the user.
- message boxes are used to display information to the user or to prompt for a decision. There are two primary ways to create a message box: using the MsgBox function or the MessageBox.Show method.
- Common Options for MessageBox.Show() OK, OKCancel, YesNo, YesNoCancel, RetryCancel, AbortRetryIgnore.
- MessageBoxIcon Information, Warning, Error, Question.

❖ Input Box

- The **Inputbox** prompts the users to enter values. After entering the values, if the user clicks the OK button or presses ENTER on the keyboard, the InputBox function will return the text in the text box. If the user clicks the Cancel button, the function will return an empty string.
- **InputBox** is a simple way to prompt the user to enter some text or a value at runtime. It displays a dialog box with a prompt message, a text box for input, and **OK / Cancel** buttons.
- InputBox(Prompt, Title, DefaultResponse, XPos, YPos)
- **Prompt** → The message shown to the user. **Title** (Optional) → The title of the

InputBox window. **DefaultResponse** (Optional) → The default text inside the text box. **XPos, YPos** (Optional) → The position of the InputBox window on the screen.